

VERIFICATION PLAN

Parameterized Systolic Array Matrix Multiplication Accelerator

Milestone 5 - UVM Verification Environment

Document Information	
Document ID	VPL-SA-MM-001
Project / Product	Parameterized Systolic Array Matrix Multiplication Accelerator
Design Version	RTL v2.0 / Milestone 4 UVM Testbench
Verification Level	Block-level verification
Prepared By	Nikhil Swarna,Hanisha Dhananjaya Produtur, Venkat Sai Sumanth Koyada
Reviewed By	[Name, Role]
Approved By	Instructor approval Pending
Date Created	2026-04-20
Last Revised	2026-05-28
Revision	1.3 - Milestone 4 UVM Update
Status	Draft / In Review
Confidentiality	☒ Public ☐ Internal ☐ Confidential ☐ Restricted

Purpose Statement

This Verification Plan defines the strategy, environment, tools, and completion criteria for the functional verification of the Parameterized Systolic Array Matrix Multiplication Accelerator.

Table of Contents

Table of Contents.....	2
1. Introduction.....	2
1.1 Purpose and Scope.....	3
1.2 Design Overview.....	3
1.3 Verification Hierarchy.....	3
1.4 References.....	3
1.5 Definitions, Acronyms, and Abbreviations.....	3
2. Functions to Be Verified.....	4
3. Functions Not Covered at This Verification Level.....	5
4. Verification Strategy and Methods.....	5
4.1 Overall Strategy.....	5
4.2 Testbench Architecture.....	5
4.2.1 Testbench Components.....	5
4.3 Stimulus Methods.....	6
4.3.1 Directed Tests.....	6
4.3.2 Constrained-Random Tests.....	6
4.3.3 Assertion-Based Verification.....	6
4.3.4 Formal Verification.....	6
4.4 Checking Strategy.....	6
5. Verification Environment.....	7
5.1 Simulation Environment.....	7
5.2 Coverage Plan.....	7
6. Required Tools.....	7
7. Completion Criteria.....	8
7.1 Simulation Completion Criteria.....	8
7.2 Bug injection Completion Criteria.....	9
7.3 Sign-Off Checklist.....	9
8. Resources and Schedule.....	9
8.1 Human Resources.....	9
8.2 Hardware Resources.....	10
8.3 Schedule.....	10
9. Risk and Mitigation.....	10
10. Document Revision History.....	11

1. Introduction

1.1 Purpose and Scope

This Verification Plan (VP) describes the complete approach for functionally verifying the Parameterized Systolic Array Matrix Multiplication Accelerator. It establishes the verification scope, methods, environment, required tools, completion criteria, and resource plan for the design under verification (DUV). The DUV is the top-level module `mm_accelerator_top`, composed of three sub-blocks: a controller FSM, an input-skew pipeline, and an $N \times N$ systolic array of processing elements (PEs).

This document applies to the following verification level:

- ☒ Block-level verification
- ☒ Sub-system verification
- ☐ Full-chip / Top-level verification
- ☐ System-level verification

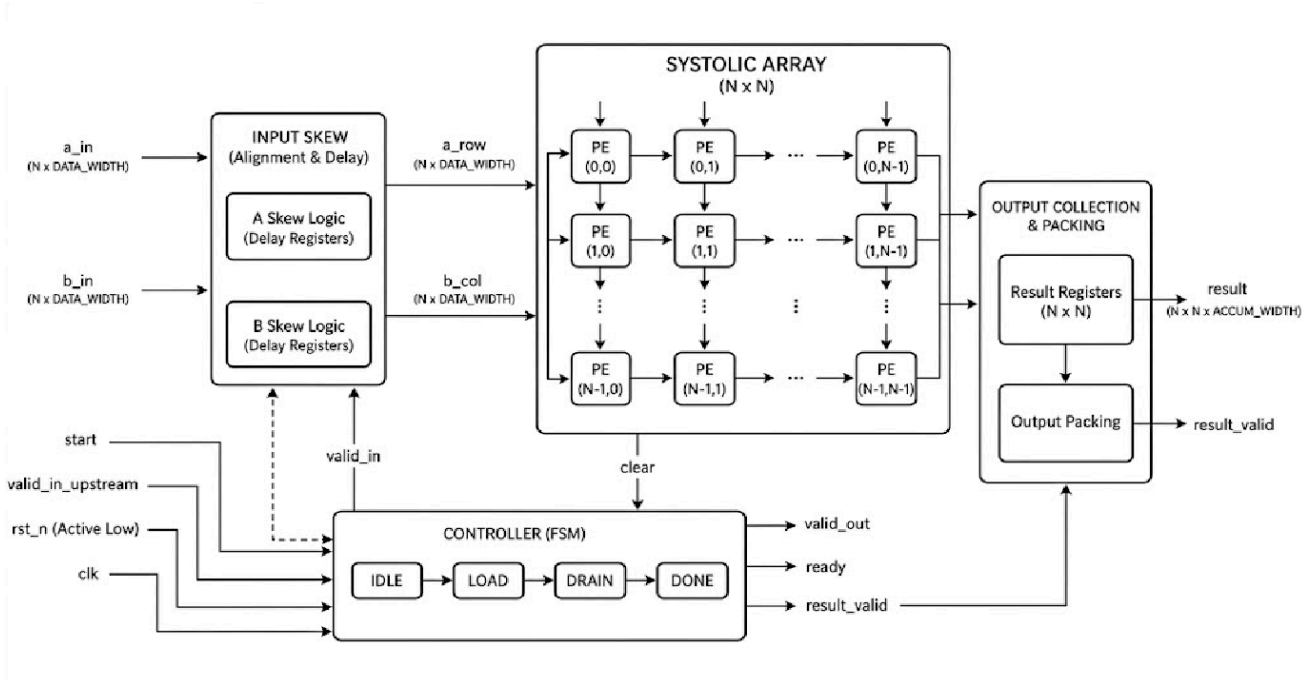
1.2 Design Overview

The Parameterized Systolic Matrix Multiplication Accelerator is a scalable hardware design implemented in SystemVerilog to efficiently perform matrix multiplication using a parallel systolic array architecture. Matrix multiplication is a fundamental and compute-intensive operation used in applications such as machine learning, signal processing, and scientific computing, where high performance is essential.

The design utilizes an $N \times N$ grid of Processing Elements (PEs), where each PE performs a multiply-accumulate (MAC) operation. Data flows through the array in a structured manner: elements of matrix A propagate horizontally, while elements of matrix B propagate vertically. As data moves through the array, each PE computes partial products and accumulates results over multiple clock cycles, enabling high parallelism and throughput.

To ensure correct timing and alignment of inputs, an input skewing mechanism is implemented using delay registers. A controller based on a finite state machine (FSM) manages different phases of operation, including data loading, computation, and pipeline draining, while generating control signals such as `valid`, `ready`, and `result_valid`.

The accelerator is fully parameterized, allowing configuration of array size and data precision, making it adaptable to various performance requirements. A UVM-based verification environment is used to validate functionality through constrained random testing and comparison with a reference model. Overall, the design demonstrates an efficient and scalable approach to hardware-accelerated matrix multiplication.



1.3 Verification Hierarchy

Verification of this design follows a hierarchical approach consistent with Bruce Wilde's Comprehensive Functional Verification framework. Each level of the hierarchy addresses a specific set of functions and feeds results to the next level.

Hierarchy Level	Design Scope	Covered In This VP	Referenced VP
Block	pe, input_skew, controller, systolic_array	[x] Yes [] No	-
Sub-system	mm_accelerator_top	[x] Yes [] No	-

1.4 References

Ref #	Document Title	Version	Date
[R01]	Design Specification – Parameterized Systolic Array Matrix Multiplication Accelerator	[X.X]	[2026-04-20]
[R02]	Comprehensive Functional Verification – Bruce Wilde	—	—
[R03]	High-throughput systolic array-based accelerator for hybrid transformer-CNN networks	[X.X]	[2018-12-30]
[R04]	Systolic Array Based Accelerator and Algorithm Mapping for Deep Learning Algorithms	[X.X]	[2024-08]

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
DUV	Design Under Verification
DUT	Device Under Test
UVM	Universal Verification Methodology
SV	SystemVerilog
TB	Testbench
FC	Functional Coverage
CC	Code Coverage
CR	Coverage Report
VP	Verification Plan
PE	Processing Element — single MAC unit in the systolic array
MAC	Multiply-Accumulate operation: $acc \leftarrow acc + a*b$
FSM	Finite State Machine (controller module)
UVM	Universal Verification Methodology (Accellera / IEEE 1800.2)
TB	Testbench
N	Array dimension parameter (default 4 → 4×4 array, 16 PEs)
DATA_WIDTH	Bit-width of A/B operands (default 8)
ACCUM_WIDTH	Bit-width of accumulator and result element (default 32)
DRAIN_CYCLES	Controller parameter — 2*N cycles of pipeline drain after last input

2. Functions to Be Verified

This section lists all functions and features of the DUV that are within scope of this verification plan, as required by the verification hierarchy (Wilde, Chapters 3–4). Each function is assigned a unique identifier for traceability.

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-001	Reset behavior — all registers (PE accum, skew delays, controller FSM, counters) reset to 0 on active-low rst_n	pe.sv, controller.sv, input_skew.sv	Critical	Directed
FV-002	Controller FSM transitions — IDLE→LOAD on start, LOAD→DRAIN after N valid inputs, DRAIN→DONE after DRAIN_CYCLES, DONE→IDLE after one cycle	controller.sv	Critical	Directed + FSM Coverage

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-003	Start-pulse handshake — start pulse in IDLE asserts clear for one cycle and arms the pipeline	controller.sv	Critical	Directed
FV-004	Input skew correctness — lane i delayed by exactly i cycles before entering the systolic array	input_skew.sv	Critical	Directed + Scoreboard
FV-005	PE MAC operation — $\text{accum} \leftarrow \text{accum} + a_in * b_in$ when <code>valid_in</code> , hold on <code>!valid_in</code> , clear to 0 on clear	pe.sv	Critical	Directed + Constrained-Random
FV-006	PE data forwarding — $a_in \rightarrow a_out$ and $b_in \rightarrow b_out$ registered by one clock	pe.sv	High	Directed
FV-007	Valid-signal diagonal propagation — <code>valid_diag[i][j]</code> follows the systolic diagonal with 1-cycle delay per hop	systolic_array.sv	High	Directed
FV-008	Full matrix multiplication correctness — $\text{result}[i][j] == \sum A[i][k] * B[k][j]$ for all (i,j) and all valid operand ranges	All modules	Critical	Scoreboard vs. reference model
FV-009	Result-valid strobe — <code>result_valid</code> pulses exactly one cycle and result bus is stable when asserted	controller.sv, mm_accelerator_top.sv	Critical	Directed
FV-010	Back-to-back operation — after DONE, a new start pulse cleanly begins a fresh matmul with reset accumulators	controller.sv + pe.sv (clear)	High	Directed + Random
FV-011	Corner values — A=0, B=0, A=Identity, B=Identity, A=all-ones, B=all-ones, max operand (0xFF)	mm_accelerator_top.sv	Critical	Directed (existing 4 tests)
FV-012	Reset behavior — all registers (PE accum, skew delays, controller FSM, counters) reset to 0 on active-low <code>rst_n</code>	pe.sv, controller.sv, input_skew.sv	Critical	Directed
FV-013	Controller FSM transitions — IDLE→LOAD on start, LOAD→DRAIN after N valid inputs, DRAIN→DONE after DRAIN_CYCLES, DONE→IDLE after one cycle	controller.sv	Critical	Directed + FSM Coverage
FV-014	Start-pulse handshake — start pulse in IDLE asserts clear for one cycle and arms the pipeline	controller.sv	Critical	Directed

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-015	Input skew correctness — lane <i>i</i> delayed by exactly <i>i</i> cycles before entering the systolic array	input_skew.sv	Critical	Directed + Scoreboard
FV-016	Back-to-back transaction readiness after result_valid and return to IDLE.	controller.sv	High	20 sequential transactions in UVM test
FV-017	No unexpected UVM errors, warnings, or fatal messages during final regression.	sim.log	Critical	UVM report summary

Priority legend: Critical = must pass before tape-out; High = required for signoff; Medium = best-effort; Low = stretch goal.

3. Functions Not Covered at This Verification Level

The following functions are explicitly excluded from this plan because they will be verified at a different level of the verification hierarchy. A corresponding Verification Plan reference is provided for each item.

NC-ID	Function / Feature	Rationale for Exclusion	Verified In (VP Reference)
NC-001	100% toggle coverage on all internal RTL signals	Some internal signals and inactive control paths do not toggle during normal matrix multiplication operations and are not forced artificially	--
	100% line coverage	Bug injected at RTL level using ifdef which is not covered in the general case scenario.	

4. Verification Strategy and Methods

This section defines the overall strategy and the specific test and checking methods that verification engineers will employ, as described in Wilde Chapters 3–4 and 7–12.

4.1 Overall Strategy

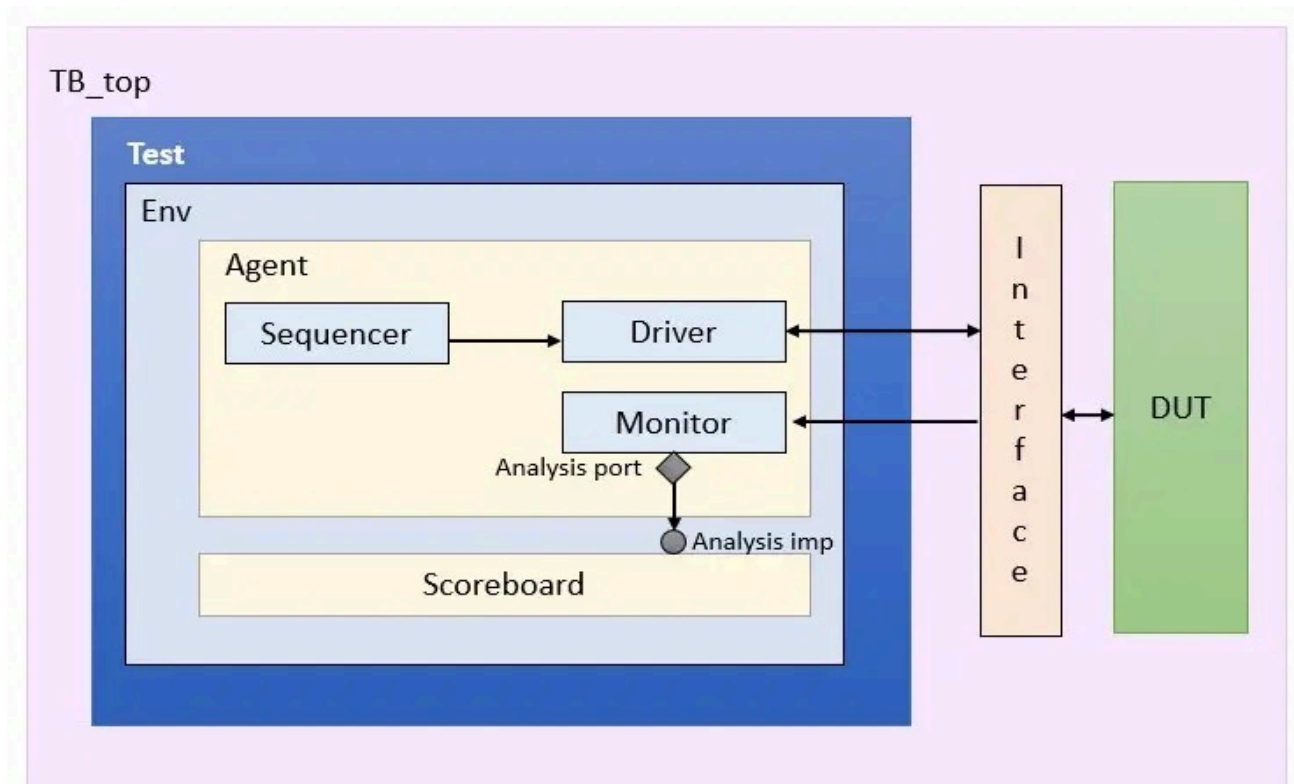
The Milestone 4 strategy uses a UVM layered testbench around the mm_accelerator_top DUT. Stimulus is generated as transaction-level matrix objects, converted by the driver into the DUT cycle protocol, observed by the monitor, and checked by the scoreboard against a behavioral reference model. The regression combines directed corner sequences with constrained-random matrix sequences. VCS code coverage is enabled for line, branch, condition, FSM, and toggle metrics.

- Directed tests for reset, zero matrix, identity matrix, and all-maximum operand cases.
- Constrained-random stimulus for 8-bit matrix values with weighted corner selection.
- Scoreboard checking using a behavioral $C = A \times B$ reference model.

- UVM phasing, objections, factory creation, sequencer-driver handshaking, monitor analysis port, and scoreboard analysis implementation.
- Code coverage collection through VCS.

4.2 Testbench Architecture

The UVM testbench is instantiated from `tb_top.sv`. The top module generates `clk`, instantiates the `mm_uvm_if` interface and DUT, sets the virtual interface into the `uvm_config_db`, and calls `run_test`. The UVM hierarchy is `mm_test` -> `mm_env` -> `mm_agent` -> `sequencer/driver/monitor`, with a separate scoreboard connected to the monitor analysis port. The design is intentionally small and readable for course use while following standard UVM structure.



Expected Topology Output:

```

uvm_test_top [mm_test]
|
|--- env [mm_env]
|   |
|   |--- agnt [mm_agent]
|   |   |
|   |   |--- seqr
|   |   |--- drv
|   |   |--- mon
|   |   |   |
|   |   |   |--- monitor_port
|   |   |   |   |
|   |   |   |   |--- scb
|   |   |   |   |   |
|   |   |   |   |   |--- scoreboard_port
|   |   |   |   |   |
|   |   |   |   |--- cov
|   |   |   |
|   |   |--- cov
|   |
|   |--- cov
|
|--- cov

```


└─ analysis_export

Our Topology Output:

```
UVM_INFO mm_test.sv(67) @ 0: uvm_test_top [TEST_CLASS] End of Elaboration Phase – UVM Testbench Topology:
UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:
```

Name	Type	Size	Value
uvm_test_top	mm_test	–	@461
env	mm_env	–	@469
agnt	mm_agent	–	@477
drv	mm_driver	–	@641
rsp_port	uvm_analysis_port	–	@658
seq_item_port	uvm_seq_item_pull_port	–	@649
mon	mm_monitor	–	@667
monitor_port	uvm_analysis_port	–	@676
seqr	mm_sequencer	–	@518
rsp_export	uvm_analysis_export	–	@526
seq_item_export	uvm_seq_item_pull_imp	–	@632
arbitration_queue	array	0	–
lock_queue	array	0	–
num_last_reqs	integral	32	'd1
num_last_rsps	integral	32	'd1
cov	mm_coverage	–	@493
analysis_imp	uvm_analysis_imp	–	@501
sch	mm_scoreboard	–	@485
scoreboard_port	uvm_analysis_imp	–	@692

4.2.1 Testbench Components

Component	Description	Reuse / New
Top-level testbench(tb_top)	Instantiates DUV, connects clocks/resets and interfaces	Reused
mm_test	Controls test scenario; extends uvm_test. Top control - runs sequences, manages objections.	Reused
mm_env	Assembles agents, scoreboards, and coverage collectors. Wires all components together via TLM connections.	Reused
mm_agent	One per interface; contains driver, monitor, sequencer	Reused
mm_scoreboard	Compares DUV output against reference model	Reused
mm_sequence	Creates and configures seq_items for a scenario	Reused
mm_uvm_if	Bundles all DUT signals into one handle	
Reference Model	Golden behavioral model of DUV	Reused
mm_seq_item	Data packet - holds A, B, expected C, actual C	Reused
mm_sequencer	Queues and routes items from sequence to driver	Reused
mm_driver	Pulls items from sequencer, drives DUT pins cycle-by-cycle	Reused

Component	Description	Reuse / New
mm_monitor	Passively observes DUT outputs, broadcasts via analysis port	Reused
mm_coverage	Implements functional coverage groups and bins	New

4.3 Stimulus Methods

4.3.1 Directed Tests

The directed portion of the UVM test covers deterministic corner behavior before the random campaign begins. The final submitted run uses Phase A with three corner sequences, then Phase B with random sequences.

- Reset sequence: driver holds rst_n low for 5 cycles and initializes all driven inputs to zero.
- Zero-A sequence: A is all zeros, so expected C is the zero matrix regardless of B.
- Identity-A sequence: A is the identity matrix, so expected C equals B.
- Maximum-value sequence: A and B use 8-bit all-ones values to stress accumulation width and large output values.

4.3.2 Constrained-Random Tests

Random stimulus is created inside mm_seq_item. A and B are randomized over the legal 8-bit range, and corner_mode uses a weighted distribution:

- 70% normal random, 10% zero-A, 10% identity-A, and 10% all-max A/B.
- The Milestone 5 test runs 17 random sequence items after the directed phase, giving 20 total matrix transactions.

4.4 Checking Strategy

Result checking will use the following methods:

- Scoreboard-based checking compares every matrix result element against a SystemVerilog behavioral reference model.
- The reference model computes $C[r][c] = \text{sum}(A[r][k] * B[k][c])$ using ACCUM_WIDTH arithmetic.
- The monitor captures the DUT packed result bus only when result_valid is asserted.
- The final UVM report requires zero UVM_ERROR and zero UVM_FATAL counts.
- The final scoreboard report must show all transactions and all matrix elements passing.

5. Verification Environment

This section describes the simulation and verification environment, in accordance with Wilde Chapters 3, 4, and 7–12.

5.1 Simulation Environment

Simulation Environment Details	
Simulator	Synopsys VCS
Simulator Version	VCS V-2023.12-SP2 Full64, as shown in the simulation log

Verification Methodology	UVM 1.1 via -ntb_opts uvm-1.1
HDL Language	SystemVerilog, compiled with -sverilog and -timescale=1ns/1ps
Assertion Language	SystemVerilog Assertions recommended; not implemented in submitted UVM package
Compile Options	-full64 -sverilog -debug_access+all -kdb +define+FSDB +incdir+. -ntb_opts uvm-1.1 -cm line+branch+cond+fsm+tgl
Simulation Options	+ntb_random_seed=42 +UVM_VERBOSITY=UVM_MEDIUM +UVM_TESTNAME=mm_test -cm line+branch+cond+fsm+tgl
Seed Management	Default makefile seed is SEED=42; random seeds can be supplied by overriding SEED on the make command line
Regression Framework	Makefile-based compile, simulate, coverage, clean, and wave targets
Waveform	FSDB waveform enabled with +define+FSDB and viewable in Verdi

5.2 Coverage Plan

Code coverage is enabled in the Milestone 4 makefile for line, branch, condition, FSM, and toggle coverage. The UVM submission primarily demonstrates stimulus and scoreboard correctness; explicit UVM functional coverage covergroups are recommended as the next closure step. The coverage plan below defines the intended functional coverage model and maps coverage goals to the functions in Section 2.

FV-ID	Coverage Group Name	Bins / Cross Points	Target (%)	Tool
FV-001	cg_corner_mode	4 bins: stimulus mode distribution	100	VCS / UVM
FV-002	cg_a_vals	6 bins: A matrix element value ranges (sampled $\times 16$)	100	VCS / UVM
FV-003	cg_b_vals	6 bins: B matrix element value ranges (sampled $\times 16$)	100	VCS / UVM
FV-004	cg_result_vals	4 bins: expected C output ranges (max = $4 \times 255^2 = 260100$)	100	VCS / UVM
FV-005	cg_fsm_signals	ready/rv/start states + ready $\times$ rv cross (run_phase clk)	100	UVM

Functional Coverage:

```
Testbench Group List
Total Groups Coverage Summary
SCORE INST SCORE WEIGHT
100.00 100.00 1

Total groups in report: 5
=====
SCORE INSTANCES WEIGHT GOAL AT LEAST PER INSTANCE AUTO BIN MAX PRINT MISSING COMMENT NAME
100.00 100.00 1 100 1 1 64 64 $unit::mm_coverage::cg_fsm_signals
100.00 100.00 1 100 1 1 64 64 $unit::mm_coverage::cg_corner_mode
100.00 100.00 1 100 1 1 64 64 $unit::mm_coverage::cg_a_vals
100.00 100.00 1 100 1 1 64 64 $unit::mm_coverage::cg_b_vals
100.00 100.00 1 100 1 1 64 64 $unit::mm_coverage::cg_result_vals
```

Code Coverage:

```
Version: V-2023.12-SP2
Command line: urg -dir uvm_cov_dir.vdb -format both -report uvm_cov_report
Number of tests: 1
```

Total Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	BRANCH	GROUP
94.90	98.97	100.00	70.41	100.00	100.00	100.00

Hierarchical coverage data for top-level instances

SCORE	LINE	COND	TOGGLE	FSM	BRANCH	NAME
93.88	98.97	100.00	70.41	100.00	100.00	tb_top

Total Module Definition Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
92.58	96.88	100.00	66.04	100.00	100.00

Total Groups Coverage Summary

SCORE	INST	SCORE	WEIGHT
100.00	100.00		1

6. Required Tools

This section lists all software tools required to support the described verification environment, consistent with Wilde Chapters 5–6. Items marked [Procure] may drive the software procurement team; items marked [Internal] require support from internal EDA/flow teams.

Category	Tool / Package	Version	Vendor / Source	Status
Simulator	VCS	V-2023.12-SP2 Full64	Synopsys	Licensed / Available
UVM Library	UVM 1.1	Included with VCS	Synopsys	Available
Coverage Merge / Report	URG / VCS Coverage	VCS 2023.12-SP2	Synopsys	Available
Waveform Debug	Verdi + FSDB	X-2025.06-SP2-1 or available campus install	Synopsys	Available
Regression	Makefile	Project-local	Internal project flow	Available
Version Control	Git / GitHub	Project repository	GitHub	Available

Category	Tool / Package	Version	Vendor / Source	Status
Documentation	DOCX/PDF reports	Course template	Project team	Available
Simulator	VCS	V-2023.12-SP2 Full64	Synopsys	Licensed / Available

7. Completion Criteria

Verification is declared complete when ALL of the following measurable criteria have been satisfied, in accordance with Wilde Chapter 13. Each criterion must be formally signed off by the Verification Lead and Design Owner.

7.1 Simulation Completion Criteria

#	Criterion	Target Threshold	Achieved Value
1	All directed corner tests pass with zero unexpected failures	100% pass rate	Met: zero-A, identity-A, and all-max sequences completed
2	Constrained-random regression passes	17 random transactions after directed phase	Met in submitted sim.log
3	Total matrix transactions pass	20 total transactions	Met: PASS=20, FAIL=0
4	All result elements pass scoreboard comparison	N*N per transaction; 320 total elements for 20 transactions	Met: PASS=20, FAIL=0
5	UVM errors and fatals	0 UVM_ERROR, 0 UVM_FATAL	Met: 0 errors, 0 fatals
6	UVM warnings	0 unexpected warnings	Met: 0 warnings in final summary
7	Code coverage metrics enabled	line+branch+cond+fsm+tgl monitored	Met: sim.log confirms monitoring; percentage report pending URG dashboard
8	Functional coverage closure	>= 95% planned functional coverage	Met: explicit UVM covergroups/subscriber not present in UVM_TB
9	Waveform generated for debug	FSDB generated when FSDB define is active	Met: makefile supported
10	No Critical or High open bugs	0 open blocking bugs	Met for submitted regression; no failing transactions reported

7.2 Bug Injection Completion Criteria

ID	Location	Category	Primary checker exercised
BUG1	controller.sv	RTL timing	Scoreboard (last-element mismatch)

BUG2	pe.sv	RTL state	Scoreboard (residual-accumulation mismatch)
BUG3	mm_monitor.sv	UVM data path	Scoreboard (transpose mismatch)
BUG4	mm_seq_item.sv + sequence	Reference model	Scoreboard (systematic mismatch)

7.3 Sign-Off Checklist

Sign-Off Item	Responsible Party	Date	Sign
Simulation completion criteria met	Verification Lead	TBD	—
Scoreboard results archived	Verification Engineer	TBD	—
Coverage report generated and reviewed	Verification Lead	TBD	—
Coverage waivers documented if needed	Design Owner / Verification Lead	TBD	—
Verification plan reviewed and up-to-date	Verification Lead	TBD	—
Final sign-off approved	Instructor / TA	TBD	—

8. Resources and Schedule

This section defines the human, hardware, and software resources required, and provides schedule estimates to tie the verification plan to program management, as required by Wilde's framework.

8.1 Human Resources

Role	Name / Team	Allocation (FTE)	Responsibilities
Verification Engineer (TB)	Hanisha	33%	Mm_accelerator_top.sv, systolic_array.sv, Mm_env.sv, tb_top.sv, mm_scoreboard.sv, mm_test.sv
Verification Engineer (TB)	Sai Sumanth	33%	pe.sv, input_skew.sv, mm_monitor.sv, mm_agent.sv, mm_coverage.sv, mm_seq_item.sv
Verification Engineer (TB)	Nikhil	33%	controller.sv, mm_driver.sv, mm_uvm_if.sv, mm_sequence.sv, mm_sequencer.sv
Instructor / Approver	Venkatesh Patil	—	Milestone review, sign-off

8.2 Hardware Resources

Resource	Specification	Quantity	Availability
PSU linux lab servers	Multi-core CPU, 32 GB RAM per host	Shared	☒ Available
VCS/ Verdi licenses	Floating license pool	Shared	☒ Available
GitHub repository	Public GitHub repo, unlimited CI minutes for public repos	1	☒ Available
Personal laptops	For documentation / editing	3	☒ Available

8.3 Schedule

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
M1	Design spec, Verification Plan, RTL, Traditional TB (this document)	[04/11]	[04/19]	All	Done
M2	Class-based TB (CLASS_TB),	[04/24]	[04/29]	V-Eng	Done
M3	Class-based TB with scoreboard + basic coverage	[04/27]	[04/29]	V-Eng	Done
M4	UVM environment — agents, sequences, uvm_env	[05/01]	[05/12]	V-Eng	Done
M5	UVM environment and coverage closure	[05/17]	[05/26]	V-Lead	☐ Open

9. Risk and Mitigation

#	Risk Description	Likelihood	Impact	Mitigation Plan
R1	Functional coverage is not fully implemented in the UVM_TB package.	Medium	High	Add a UVM coverage subscriber or covergroups for corner_mode, operand ranges, controller states, result_valid behavior, and result ranges.
R2	Coverage percentages are unavailable if URG report is not archived.	Medium	Medium	Run make all or make cov and archive uvm_cov_report/dashboard.html, cov.log, and summary metrics.
R3	Random regression may miss rare corner combinations.	Medium	Medium	Keep deterministic directed corner sequences and add seed regression with coverage-guided constraints.
R4	Packed bus indexing mistakes can hide row/column ordering bugs.	Medium	High	Maintain monitor unpacking checks and compare all N*N elements against reference model.
R5	Controller timing bugs may appear in back-to-back operation.	Medium	High	Run sequential transaction stress, check result_valid-to-next-start

#	Risk Description	Likelihood	Impact	Mitigation Plan
				behavior, and add SVA for controller transitions.
R6	Tool license or environment issue delays regression.	Low	Medium	Keep Makefile flow simple and archive reproducible logs from known-good VCS setup.

10. Document Revision History

Revision	Date	Author	Description of Change	Approved By
0.1	2026-04-20	Project team	Initial verification plan draft	--
1.1	2026-04-30	Project team	Milestone 3 / class-based verification update	--
1.2	2026-05-14	Project team	Milestone 4 / UVM – based verification.	--
1.3	2026-05-28	Project Team	Updated plan for Milestone 5 UVM environment, UVM components, simulation results, and pending coverage closure items	--

End of Verification Plan | Parameterized Systolic Array Matrix Multiplication Accelerator | VPL-SA-MM 001